Performance Analysis

assign251_2 — Log4j 2 custom appender and Velocity layout
Student ID: 25010416

Purpose and method

This report compares ArrayList- and LinkedList-backed MemAppender configurations, Log4j 2 ConsoleAppender and FileAppender, and PatternLayout and the custom VelocityLayout. VisualVM captures provide supporting observations about heap use, live objects and CPU sampling.

The latest MemAppender benchmark used 100,000 append iterations per measured round, five measured rounds, and maximum stored-log limits of 100, 1,000 and 10,000. Reported timing and allocation values are medians across those five rounds. The layout comparison formatted 20,000 messages over five rounds (median). The appender comparison logged 100 messages and reports one measured result per appender. VisualVM captures came from a separate ProfilingWorkload run.

MemAppender: timing and allocation results

maxSize	ArrayList time	LinkedList time	ArrayList allocated	LinkedList allocated
100	2.874 ms	1.992 ms	2.1 KiB	2,344.5 KiB
1,000	11.191 ms	2.314 ms	15.4 KiB	2,344.5 KiB
10,000	83.002 ms	2.050 ms	165.8 KiB	2,344.5 KiB

Allocation metric: allocated bytes on the benchmark thread during the append loop, measured using the JDK thread-allocation management extension. This is not retained heap size and does not measure memory owned exclusively by the list. Results are JVM- and workload-specific.

LinkedList was faster in this implementation because reaching maxSize repeatedly removes the oldest item. Removing index zero from an ArrayList shifts remaining elements, while removing the first node from a LinkedList avoids that shift. The allocation result also differs: the LinkedList workload allocates nodes for appends, whereas ArrayList can reuse its backing array until it needs to grow. These observations apply to this bounded-eviction workload, not every use of the collections.

Other timing results

Configuration	Workload	Measured time	Interpretation
PatternLayout	20,000 messages; median of 5	2.337 ms	Faster in this formatting test.
VelocityLayout	Same workload	956.874 ms	About 409.5× PatternLayout time in this run.
ConsoleAppender	100 messages; one measured result	5.131 ms	Slower in this single run.
FileAppender	Same workload	1.021 ms	Faster in this single run; output was 2,790 bytes.

VelocityLayout took substantially longer than PatternLayout. Template evaluation and per-event context setup are plausible contributors, but this benchmark does not isolate the cause. The ConsoleAppender/FileAppender result favoured FileAppender in one run only; console, operating-system and filesystem conditions can influence the result.

VisualVM observations

The Monitor capture shows about 38.3 MB used heap at that moment, with a displayed heap size of about 272.6 MB and 14 live threads. The Memory Sampler shows byte[] as the largest sampled live-object category (22,781,944 bytes; 62.6%), followed by String (2,890,632 bytes; 7.9%). These are sampled live-object estimates and do not prove that every byte array belongs to MemAppender.

The CPU Sampler shows getEventStrings() at 75.8% of displayed sampled CPU time (20,020 ms accumulated profiler time), suggesting event formatting is a potential hotspot. This is cumulative profiler time, not one call's latency. The separate CPU Profiler capture reports 1,368 ms CPU time overall; clipped method names limit precise attribution. These profiler figures should not be compared directly with short JUnit timings.

Limitations and conclusion

The results come from one local machine and Java/Maven environment. MemAppender and layout timings use medians from five rounds, while the appender comparison is a single measured result. Thread-allocation figures describe bytes allocated by the benchmark thread during the append loop, not retained heap size; they are affected by JVM implementation and workload. VisualVM images are snapshots rather than controlled memory experiments. JVM warm-up, garbage collection, background activity and I/O conditions may affect results. In this workload, LinkedList was faster for bounded eviction, PatternLayout was much faster than VelocityLayout, and the single appender comparison favoured FileAppender. VisualVM highlighted event formatting as a sampled CPU cost and byte arrays as the largest sampled live-memory category.

The latest test run completed successfully: 22 tests, zero failures, zero errors. The JaCoCo report is generated by Maven under target/site/jacoco.

VisualVM evidence (1/4)

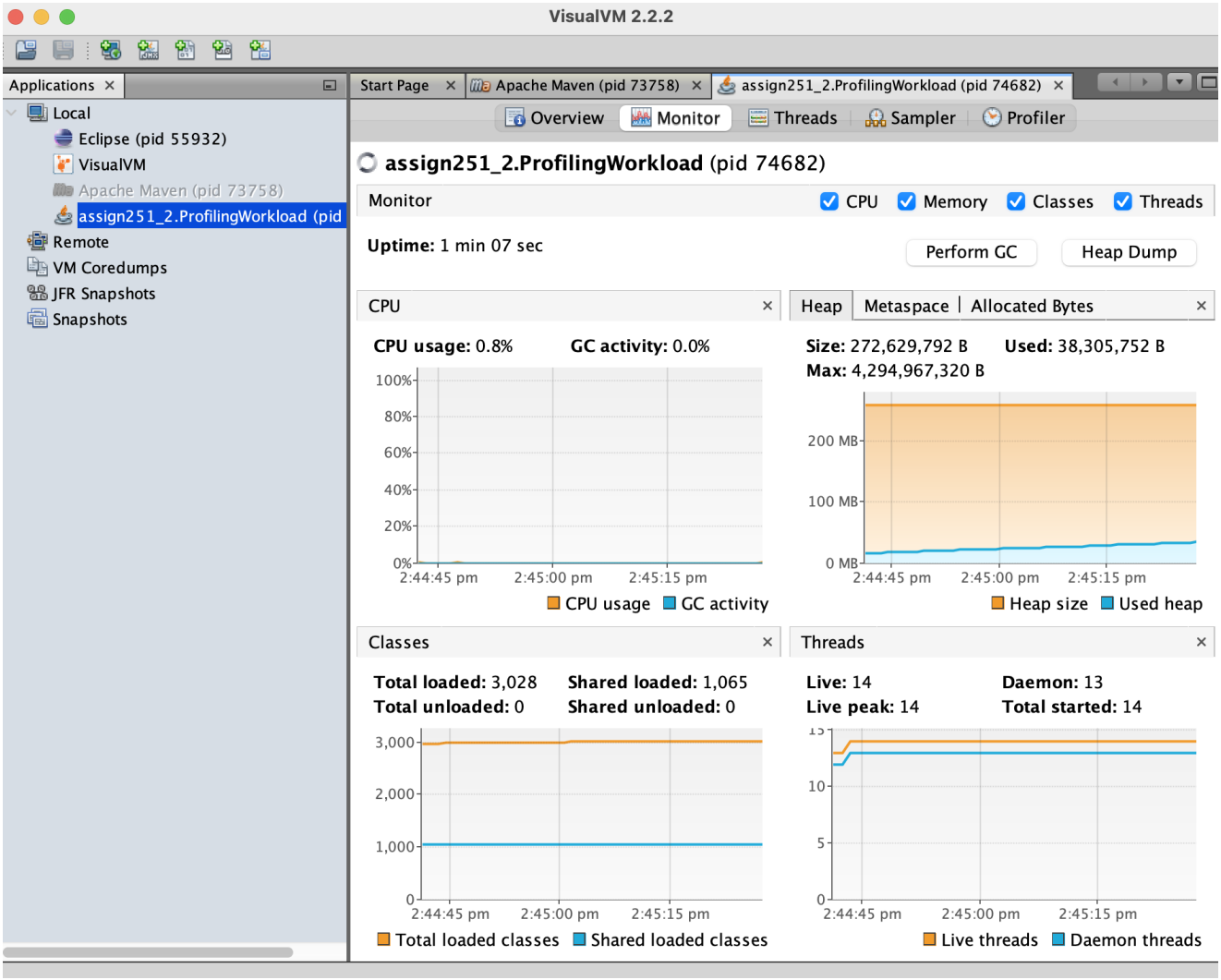


Figure 1. VisualVM Monitor overview. At capture time, used heap was about 38.3 MB, displayed heap size about 272.6 MB, and 14 live threads were reported.

VisualVM evidence (2/4)

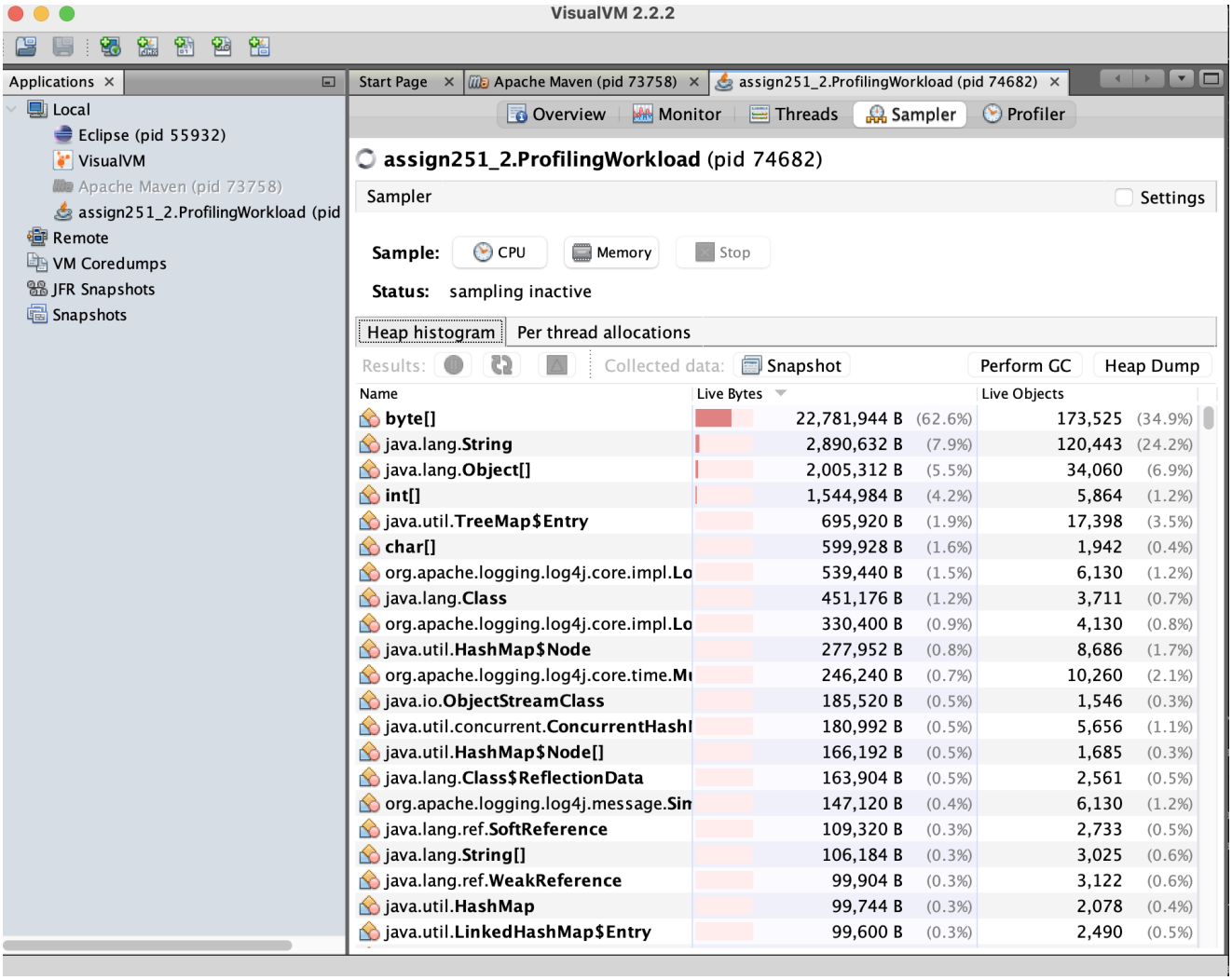


Figure 2. Memory Sampler. byte[] represented 22,781,944 bytes (62.6% of sampled live bytes), followed by String at 2,890,632 bytes (7.9%). These are sampled live-object estimates.

VisualVM evidence (3/4)

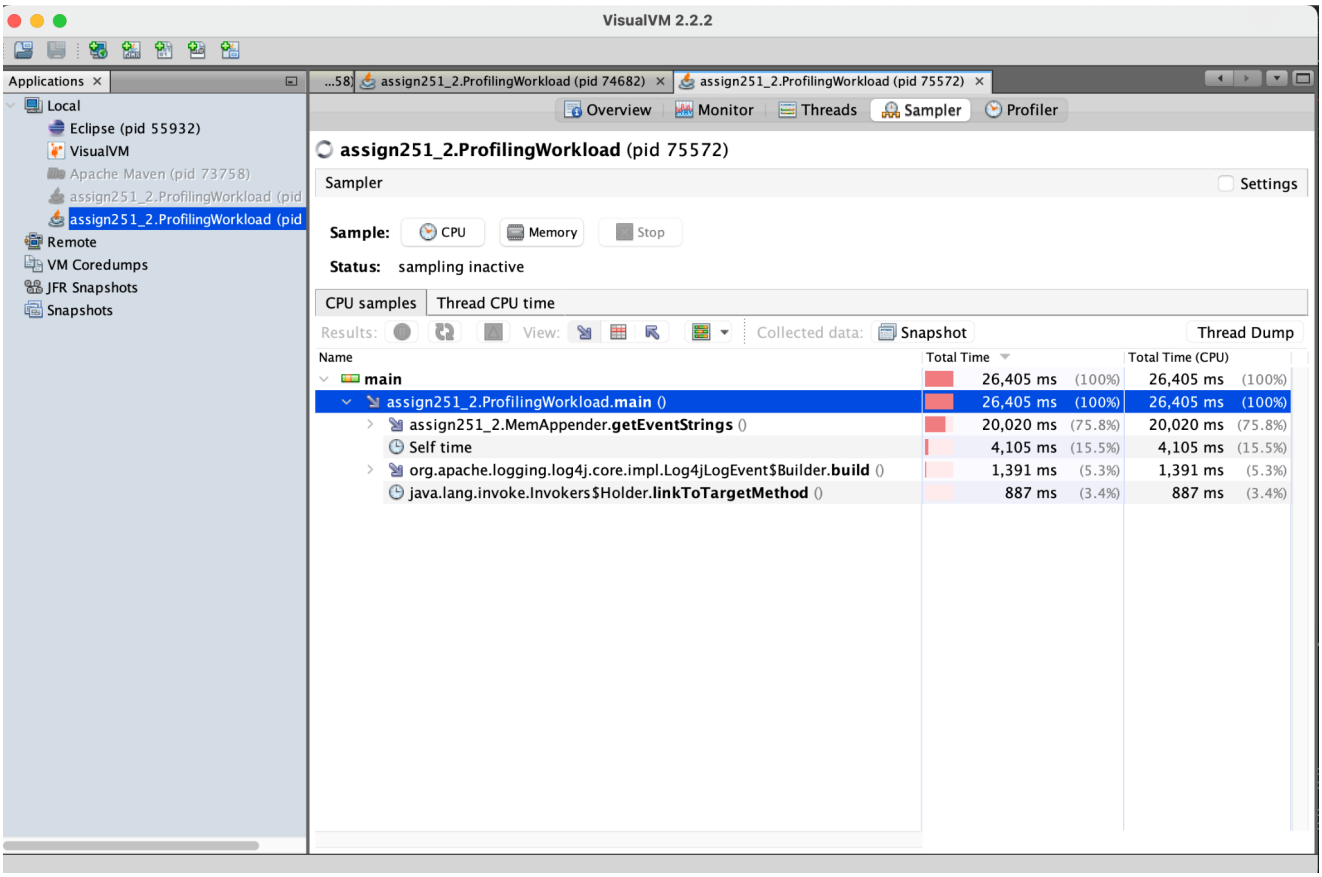


Figure 3. CPU Sampler. getEventStrings() accounted for 20,020 ms (75.8%) of displayed sampled CPU time. This is cumulative profiler time, not one call's latency.

VisualVM evidence (4/4)

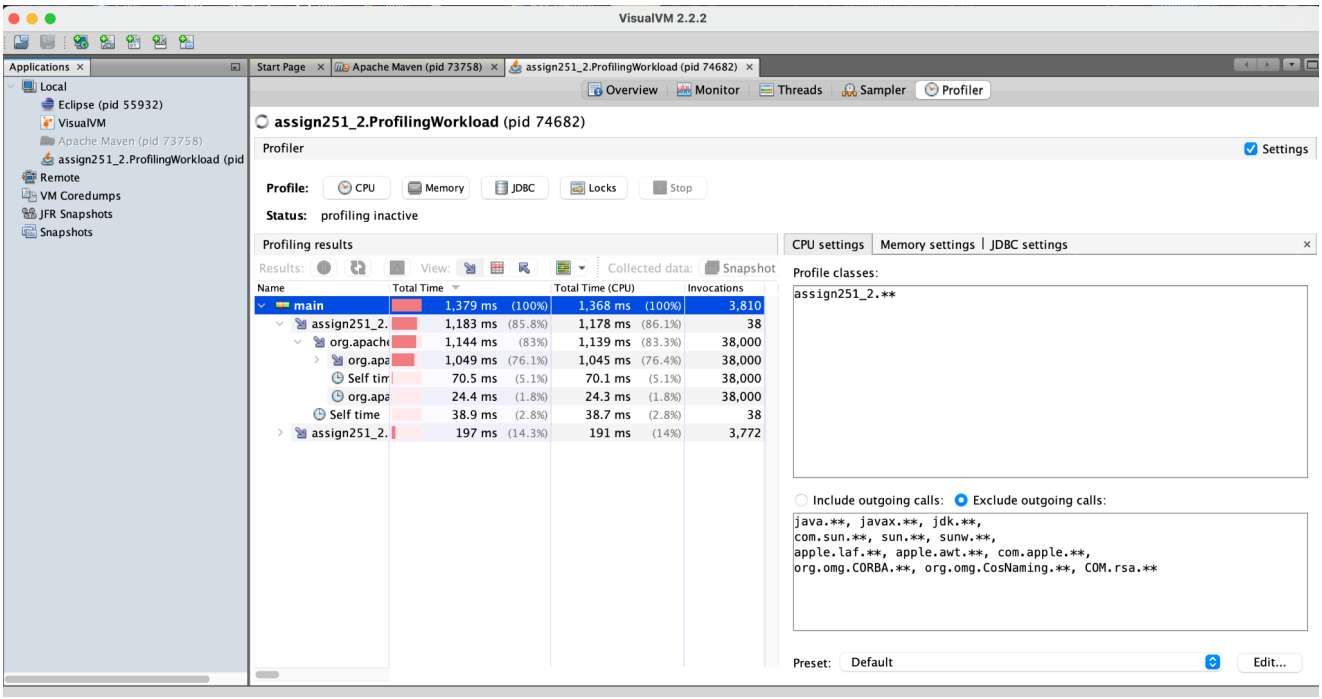


Figure 4. CPU Profiler. The capture reported 1,368 ms CPU time overall. Clipped method names limit precise attribution.